

PRESTIX.VIP

Promoter Magic Link & Social Broadcasting

Security & Vulnerability Analysis · Implementation Roadmap

Attribute	Value
Document scope	Addendum B — Multi-Promoter Preference Attribution & Venue Performance Dashboard
Base spec	PRESTIX Magic Link & Referral Implementation Strategy v1.0
Codebase reviewed	appkit-siwe-auth-module / zenstack/schema.zmodel (live)
Analysis date	March 2026
Classification	CONFIDENTIAL — Internal Engineering
Status	Flagged Issues Resolved → Phase Plan Approved

1. Executive Summary

This document provides a complete security and vulnerability audit of the proposed Addendum B implementation, followed by a phased integration plan for the promoter magic-link generation feature and social community broadcasting capability.

The Addendum B specification (Multi-Promoter Preference Attribution & Venue Performance Dashboard) is architecturally sound. The conflict-resolution model, scope-binding hierarchy, and commission pipeline are well-reasoned. However, the analysis below identifies

seven specific implementation concerns — two at CRITICAL severity, two HIGH, and three MEDIUM — that must be resolved before beginning the phase rollout. None require design changes to the spec; all are implementation-layer gaps.

Following the flag analysis, a five-phase integration plan is presented that addresses: (1) schema additions, (2) link generation UX and API, (3) social channel pairing and broadcast, (4) venue/event nomination workflow, and (5) the performance dashboard.

2. Vulnerability & Flag Analysis

Each flag below identifies an issue in the proposed implementation spec or the existing codebase, its risk surface, and the recommended resolution.

▸ **CRITICAL: No Unique Constraint on MemberPromoterPreference Conflict Scope**

Issue

The proposed MemberPromoterPreference model (section A.5) stores conflicts identified by the combination of (memberId, venueId, eventId, scopeDate, bindingScope). The spec defines this as the canonical conflict scope, but the schema includes no @@unique constraint across these fields. Without a database-enforced unique index on this composite key, a race condition at link ingestion time (e.g., two requests arriving within milliseconds on different server instances) can create duplicate PENDING_CHOICE rows for the same scope.

Risk

If two **PENDING_CHOICE** rows are created for the same member+scope, the commission pipeline (section A.6, step A.PRE-1) will find multiple preference records, producing non-deterministic promoter selection and potential double-commission scenarios.

Recommendation

Add the following to the **MemberPromoterPreference** model before migration:

```
@@unique([memberId, bindingScope, venueId, eventId, scopeDate])
```

Use upsert with conflict-target at the middleware layer rather than raw insert. If a **PENDING_CHOICE** already exists for a scope, append the challenger to the existing record rather than creating a new one.

► **CRITICAL: PENDING_CHOICE State Blocks Checkout Without a Timeout Fallback**

Issue

Section A.4 states that when status = **PENDING_CHOICE**, the checkout is **BLOCKED**. Section A.4 also notes the fallback to **FIRST_ATTRIBUTION** when the member dismisses the prompt — but this fallback is described as UX-initiated (dismiss action). There is no server-side TTL or cron-based expiry for stale **PENDING_CHOICE** records. A member who clicks a link, generates a **PENDING_CHOICE**, then abandons the session (no dismiss, no choice) will remain permanently blocked if they return to checkout hours or days later via a direct URL.

Risk

Stuck **PENDING_CHOICE** records cause checkout abandonment and silently deny commission to both promoters indefinitely. The binding "after first transaction" lock presupposes a transaction can be completed, which it cannot while **PENDING_CHOICE** persists unresolved.

Recommendation

Add a **resolvedDeadlineAt** field (DateTime?) to **MemberPromoterPreference**. Set it to **createdAt** + 24 hours at **PENDING_CHOICE** creation. A lightweight cron job (or edge middleware check at checkout entry) should auto-resolve any record where status = **PENDING_CHOICE** AND **resolvedDeadlineAt** < now() by applying the **FIRST_ATTRIBUTION** rule. This is consistent with the spec's stated fallback intent.

► **HIGH: MemberPromoterPreference Access Policy Allows Cross-Member Read**

Issue

The existing **PromoterMagicLink** access policy (line 940 in **schema.zmodel**) correctly scopes reads to the owning promoter. However, the proposed **MemberPromoterPreference** model carries **memberId**, **primaryPromoterId**, and **challengerPromoterId**. The spec does not define a ZenStack **@@allow** policy for this model. If the default ZenStack v3 policy (deny all unless specified) is not correctly applied, an authenticated promoter querying their own magic link's referral could traverse the relation to read preference records of competing promoters or of members they did not originate — exposing cross-tenant attribution data.

Risk

Multi-tenant data leakage. A promoter could enumerate which members selected a competitor as BOUND for a venue event, undermining the member's choice confidentiality.

Recommendation

Explicitly define the following policies on MemberPromoterPreference before any relation traversal is enabled:

```
@allow('read', auth().id == member.userId || auth().role == PLATFORM_ADMIN)
```

```
@allow('create', auth().role == PLATFORM_ADMIN)
```

```
@allow('update', auth().role == PLATFORM_ADMIN)
```

Promoters must never be granted direct read access to preference records. Their performance impact is surfaced only through the aggregated PromoterVenuePerformance model, consistent with the open design question Q3 recommendation ("aggregated only").

► **HIGH: boundDecayCounter Field Missing from MemberPromoterPreference Schema**

Issue

Section A.7 defines the Extended MemberPromoterPreference Fields including boundDecayCounter (Int @default(0)), lastCodeUsedInScope, lastCodeUsedAt, totalScopeSpend, and transactionCount. These are described as critical for the scope-local decay model (section A.6, step A.PRE-3). However, the base MemberPromoterPreference schema (section A.5) does not include them — they are listed separately in A.7 without an explicit instruction to merge them. This creates an ambiguity where implementers may build the base model from A.5 and miss the A.7 extensions entirely.

Risk

The commission pipeline would apply the global MemberProfile.decayTransactionCount instead of the scope-local counter, awarding commissions to the wrong promoter after the first code-entry event in a bound scope. This directly violates the spec's intent.

Recommendation

The A.7 fields must be treated as an integral part of the MemberPromoterPreference model, not an optional extension. Consolidate into a single schema definition in the implementation. The merged model requires the following additional fields: boundDecayCounter Int @default(0), lastCodeUsedInScope String?, lastCodeUsedAt DateTime?, totalScopeSpend Decimal @db.Decimal(18,2) @default(0), transactionCount Int @default(0).

► **MEDIUM: PromoterVenueAssignment Model is Absent — Venue Nomination Flow Has No Schema Anchor**

Issue

The user's extended request (and the social broadcasting workflow) requires that a venue host or event organizer nominates specific promoters to broadcast a given event. The Addendum B spec focuses on the analytics and preference side, but neither the base spec nor the current schema.zmodel contains a PromoterVenueAssignment or VenueEventPromoter junction model. The existing schema has VenueProfile and VenueEvent but no model linking a promoter to a specific event as "accepted/nominated". The channel field on PromoterMagicLink (String? — "whatsapp" | "instagram_dm" etc.) captures the delivery channel for individual links but does not model a persistent social channel or group configuration against the promoter's profile.

Risk

Without a VenueEventPromoter assignment model, the system cannot enforce that a promoter generating a magic link for an event is actually authorised by the venue admin. Any ACTIVE promoter could generate a link for any event, polluting attribution and creating fraudulent commission claims. This is a correctness and financial integrity risk.

Recommendation

Add VenueEventPromoter (junction: venueId, eventId, promoterId, status: NOMINATED|ACCEPTED|DECLINED|SUSPENDED, nominatedBy, acceptedAt) and PromoterSocialChannel (promoterId, platform: WHATSAPP|TELEGRAM|INSTAGRAM|SMS, channelHandle, channelName, channelType: DM|GROUP|CHANNEL, isPrimary, verifiedAt). Magic link generation must validate that an ACCEPTED VenueEventPromoter record exists for the promoter+event combination before issuing the link.

► **MEDIUM: PromoterVenuePerformance Snapshot Has No isStale Flag in Schema but References it in Batch Logic**

Issue

Section B.11 batch job step 17 states: "Marks any existing rows with the same key but older snapshotDate as isStale = true." However, the PromoterVenuePerformance schema (section B.5) does not include an isStale Boolean field. The schema as written would throw a runtime error on any batch job that attempts to set this field.

Recommendation

Add isStale Boolean @default(false) and a composite @@index([promoterId, venueId, windowDays, isStale]) to PromoterVenuePerformance. Dashboard queries should filter WHERE isStale = false to return only current snapshots.

► **MEDIUM: Open Design Question 1 — Pre-transaction Binding Change Window Not Resolved**

Issue

The spec's open design question Q1 asks whether a member can change their BOUND promoter before any transaction has been processed. The recommendation is Option A (allow change until first transaction). This is the correct UX decision, but it creates an implementation ambiguity: the `isLocked` field on `MemberPromoterPreference` is set "after first transaction" per section A.2 step 8/`isLocked`. If the member changes their bound promoter pre-transaction, the old BOUND record must be correctly moved to DECLINED and the new selection must become BOUND — but the `FIRST_ATTRIBUTION` fallback path in step A.PRE-4 does not account for this re-selection path, potentially creating two BOUND records for the same scope.

Recommendation

Add a state transition guard at the middleware level: before setting any record to `status = BOUND`, check that no other BOUND (and non-expired, non-locked) record exists for the same `member+scope` composite key. If one exists and `isLocked = false`, set it to DECLINED before completing the new BOUND transition. This must be atomic (wrapped in a database transaction).

3. Summary of Required Schema Additions

The following table consolidates all new models, fields, and constraints required before beginning Phase 1 implementation:

Model / Element	Change	Reason
MemberPromoterPreference	<code>@@unique([memberId, bindingScope, venueId, eventId, scopeDate])</code>	Flag 1 — prevent race-condition duplicates
MemberPromoterPreference	<code>resolvedDeadlineAt DateTime?</code>	Flag 2 — auto-expire stale <code>PENDING_CHOICE</code>
MemberPromoterPreference	Merge A.7 fields into base model (5 fields)	Flag 4 — scope-local decay counter
MemberPromoterPreference	<code>@@allow</code> policies (member-scoped read only)	Flag 3 — prevent cross-promoter data leakage
MemberPromoterPreference	State transition guard middleware (atomic)	Flag 7 — prevent dual BOUND records
PromoterVenuePerformance	<code>isStale Boolean @default(false) + index</code>	Flag 6 — batch job alignment
VenueEventPromoter (NEW)	<code>venueId, eventId, promoterId, status, nominatedBy, acceptedAt</code>	Flag 5 — nomination/authorisation anchor
PromoterSocialChannel (NEW)	<code>promoterId, platform, channelHandle, channelType, isPrimary, verifiedAt</code>	Flag 5 — persistent social channel config

4. Phase Integration Plan

All flags in Section 2 must be resolved before Phase 1 begins. The five phases below assume a clean migration path from the current schema state.

Phase 1 — Schema Foundation & Security Hardening

Goal: Land all new models and resolve all flagged vulnerabilities into the database layer.

1.1 New Models

- Add MemberPromoterPreference model (merged A.5 + A.7) with full ZenStack access policies, unique constraint, and resolvedDeadlineAt field.
- Add PromoterVenuePerformance model (section B.5) with isStale field and composite index.
- Add PromoterPerformanceSnapshot model (section B.6).
- Add VenuePromoterWeightConfig model (section B.8).
- Add VenueEventPromoter model (new — see Flag 5 recommendation).
- Add PromoterSocialChannel model (new — see Flag 5 recommendation).

1.2 Enum Additions

- PromoterBindingScope: EVENT_DATE | VENUE_DATE | VENUE_OPEN | PLATFORM_OPEN
- MemberPreferenceStatus: PENDING_CHOICE | BOUND | BOUND_EXPIRED | DECLINED
- PromoterPerformanceDimension: CONVERSION | VELOCITY | PRE_EVENT | ON_NIGHT | GENERAL_SPEND
- VenueEventPromoterStatus: NOMINATED | ACCEPTED | DECLINED | SUSPENDED
- SocialChannelPlatform: WHATSAPP | TELEGRAM | INSTAGRAM | SMS | EMAIL

1.3 Migration & Validation

- Generate and run Prisma/ZenStack migration. Verify all unique constraints are enforced at the DB level.
- Add middleware state transition guard for BOUND uniqueness per scope (atomic upsert pattern).
- Write unit tests: conflict detection, FIRST_ATTRIBUTION fallback, scope precedence order.

Deliverable:

Migrated database, all models live, all access policies tested, zero cross-tenant read paths.

Phase 2 — Link Generation UX & API

Goal: Enable a promoter to generate a magic link scoped to a specific venue and event they have been accepted for, and preview or copy it for distribution.

2.1 Promoter Link Generator API

- POST /api/promoter/magic-links — validates VenueEventPromoter.status = ACCEPTED before creating PromoterMagicLink. Accepts presetVenueId, presetEventId, presetEventDate, channel, campaignTag.
- GET /api/promoter/magic-links — returns paginated list of the authenticated promoter's links with click/conversion aggregates.
- DELETE /api/promoter/magic-links/[id] — soft-deactivates a link (sets isActive = false, deactivatedAt).

2.2 Link Ingestion Middleware (Attribution)

- Extend existing /join/[shortCode] route to: (a) record MagicLinkClickEvent, (b) check for existing attribution for same scope, (c) if conflict detected, create PENDING_CHOICE preference record (using upsert with unique constraint), (d) store candidate attribution in session cookie for resolution at checkout.
- Implement the scope precedence resolution: EVENT_DATE > VENUE_DATE > VENUE_OPEN > PLATFORM_OPEN.
- Set resolvedDeadlineAt = now() + 24h on new PENDING_CHOICE records.

2.3 Promoter Link Dashboard UI (Hub)

- Extend /hub/promoters/[id] page to show the promoter's own links, per-link analytics (clicks, conversions, revenue), and a "Create New Link" button.
- Create link builder form: venue selector (filtered to ACCEPTED assignments), event selector, date picker, optional channel tag, campaign label, max uses, expiry.
- Link output: full URL + QR code preview + copy-to-clipboard button.

Deliverable:

Promoter can log in, see their accepted venue/event assignments, generate scoped links, and view click analytics.

Phase 3 — Social Channel Pairing & Broadcast

Goal: Allow a promoter to configure their social channels (WhatsApp groups, Telegram channels, Instagram accounts, etc.) and broadcast generated links directly to those channels with a single action.

3.1 Social Channel Configuration

- Settings page section: "My Broadcast Channels" — lists PromoterSocialChannel records.
- Add channel form: platform selector, channel handle/name, channel type (DM | GROUP | CHANNEL), set as primary toggle.
- Platform-level verification flow (lightweight): for WhatsApp/Telegram, record the channel ID; for Instagram, OAuth token or handle confirmation. Mark verifiedAt on success.

3.2 Broadcast Action from Link Generator

- After link creation (Phase 2), show a "Broadcast to Channels" panel listing all of the promoter's configured channels with checkboxes.
- Promoter selects channels and clicks "Send". The system constructs platform-appropriate share payloads:
- WhatsApp: opens wa.me share URL with pre-composed message including the link, event name, venue name, and date.
- Telegram: opens t.me/share/url with message text.
- Instagram: copies link to clipboard (Instagram API does not support programmatic DM send from web; UX note shown to user).
- SMS: opens sms: scheme with pre-composed body.
- Record each broadcast attempt in a PromoterLinkBroadcast log table (linkId, channelId, platform, broadcastAt, deliveryStatus).

3.3 Pre-configured Channel Targeting at Link Creation

- Add optional channelId field to magic link creation form — "Create link for specific channel". This pre-stamps the channel field on PromoterMagicLink and enables per-channel attribution analytics.

Deliverable:

Promoter can manage their channel list, generate a scoped link, and immediately broadcast it to one or more configured channels without leaving the platform.

Phase 4 — Venue Nomination & Promoter Acceptance Workflow

Goal: Enable venue hosts and event organizers to nominate promoters for specific events, and enable promoters to accept, giving them the right to generate links for that event.

4.1 Venue Admin — Nominate Promoters

- Venue admin dashboard: Events tab → select event → "Manage Promoters" panel showing current nominations.
- Search/invite promoter by referral code or name. Creates VenueEventPromoter record with status = NOMINATED.
- Venue admin can SUSPEND an accepted promoter mid-event (triggers re-preference prompt for any BOUND members per open design Q2 recommendation).
- Venue admin can view per-promoter nomination history and performance summary inline (links to Phase 5 dashboard).

4.2 Promoter — Review & Accept Nominations

- Notifications model already in schema. Send in-app + email notification to promoter on NOMINATED status.
- Promoter hub: "Event Invitations" section — lists pending nominations with venue name, event name, date.
- Promoter clicks "Accept" → VenueEventPromoter.status = ACCEPTED, acceptedAt = now(). Promoter is now authorised to generate links for this event.
- Promoter clicks "Decline" → status = DECLINED. Venue admin is notified.

4.3 Attribution Enforcement

- Middleware at link creation (Phase 2) verifies VenueEventPromoter.status = ACCEPTED. Any attempt to create a link for a non-accepted event returns 403 with a clear message.
- If a promoter's assignment is SUSPENDED after link creation but before the event, existing links remain valid (clicks still tracked) but the commission pipeline routes to PLATFORM at the scope level — consistent with the spec's Q2 recommended approach.

Deliverable:

End-to-end authorisation flow: venue nominates → promoter accepts → promoter generates link → attribution is gated on acceptance. No unauthorised commission claims possible.

Phase 5 — Venue Promoter Performance Dashboard

Goal: Implement the PromoterVenuePerformance analytical model, batch computation job, and the three-view dashboard UI for venue admins.

5.1 Batch Computation Job

- Implement the nightly cron job (03:00 venue local time) per section B.11.
- Processes all venueId+promoterId pairs active in the past 180 days.
- Computes all 5 raw metrics across windowDays [7, 30, 90, 180, 0] from source tables.
- Normalises to 0–100 scores, applies VenuePromoterWeightConfig, computes compositeScore, assigns venueRank.
- Upserts PromoterVenuePerformance rows; marks stale rows isStale = true.
- Also creates PromoterPerformanceSnapshot rows per event night.

5.2 API Endpoints

- GET /api/venues/[id]/promoter-performance — ranked list with filters (window, eventScope, promoterStatus, minBookings, tier).
- GET /api/venues/[id]/promoter-performance/[promoterId] — full detail + snapshot history.
- GET /api/venues/[id]/promoter-performance/[promoterId]/events — per-event snapshot rows.
- PUT /api/venues/[id]/promoter-weight-config — save custom dimension weights (validates sum = 100, triggers async recompute).
- POST /api/venues/[id]/promoter-performance/refresh — on-demand recompute, rate-limited to 1 per hour.

5.3 Dashboard UI (Three-View)

- View 1 — Promoter League Table: ranked table with composite score chips (green 80–100, amber 60–79, red below 60), sortable by any dimension, CSV export.
- View 2 — Promoter Detail Drawer: funnel chart (clicks → registrations → bookings → payments), radar chart of 5 dimension scores, event history table, commission statement, compare overlay for two promoters.
- View 3 — Event Night Breakdown: pre-event vs on-night spend split, attendance confirmation rate, avg click-to-check-in time.

5.4 Promoter Self-View (Open Design Q4 — Option B)

- Promoter hub shows their own composite score and venue rank (e.g., "#3 of 12 at Ku De Ta") but not competitor detail scores — consistent with recommended approach.

Deliverable:

Venue admins have a fully operational promoter performance dashboard enabling data-driven nomination, retention, and incentive decisions. Promoters have visibility of their own standing.

5. Phase Dependency & Timeline

Phase	Depends On	Key Output
Phase 1 — Schema & Security	Flag resolutions confirmed	Migrated DB, all models live, policies tested
Phase 2 — Link Generation	Phase 1 complete	Promoter creates, previews, copies scoped links
Phase 3 — Social Broadcast	Phase 2 complete	Promoter broadcasts links to configured channels
Phase 4 — Nomination Flow	Phase 1 complete (parallel with 2)	Venue nominates; promoter accepts; link gated on acceptance
Phase 5 — Performance Dashboard	Phases 1, 2, 4 complete	Venue admin analytics + promoter self-view

Note on Parallelism

Phase 3 (Social Broadcast) and Phase 4 (Nomination Flow) can be worked in parallel by separate engineering tracks once Phase 1 is complete. Phase 5 requires the VenueEventPromoter data from Phase 4 to produce meaningful per-event attribution snapshots, so Phase 5 should not begin until at least the Phase 4 acceptance flow is live.

6. Open Design Question Resolutions

The four open questions in Addendum B are resolved below, consistent with the spec's own recommendations:

Q#	Question	Adopted Resolution
Q1	Can member change bound promoter before first transaction?	YES — allow change until first transaction. State transition guard (Phase 1) prevents dual BOUND records.
Q2	Bound promoter suspended before event?	Option B — surface new preference prompt to member. Platform absorbs the commission gap for the transition period.
Q3	Should venue see member-level granularity?	Aggregate only to venue admin. Member-level data available to PLATFORM_ADMIN only via access policy.
Q4	Should promoter see their own performance scores?	Option B — promoter sees own score and venue rank only (e.g., "#3 of 12"). Competitor details hidden.

7. How a Promoter Generates & Broadcasts a Link

This section narrates the complete end-user experience once all five phases are implemented — specifically addressing the promoter's workflow for link generation and social distribution to paired channels and groups.

7.1 Preconditions

- Venue admin has created an event in the platform and nominated the promoter via the "Manage Promoters" panel (Phase 4).
- Promoter receives an in-app notification and email: "You've been invited to promote [Event Name] at [Venue] on [Date]."
- Promoter accepts the invitation in their hub. `VenueEventPromoter.status = ACCEPTED`.
- Promoter has previously configured at least one social channel in their Profile settings (Phase 3).

7.2 Link Generation Flow

1. Promoter navigates to Hub → My Links → Create New Link.
2. The venue selector shows only venues where the promoter has `ACCEPTED` assignments. They select the venue.
3. The event selector shows only upcoming events at that venue where `VenueEventPromoter.status = ACCEPTED` for this promoter. They select the event and date.
4. Optionally: pre-fill target name, target email, pre-select table type or ticket tier for a personalised link.
5. Set optional parameters: campaign tag, expiry date, max uses.
6. Click "Generate Link". System validates `ACCEPTED` assignment, generates cryptographically random `linkToken`, derives `shortCode`, stores `PromoterMagicLink` record with `presetVenueld`, `presetEventId`, `presetEventDate`, `referralCode` (inherited from `promoter.referralCode`).
7. Link card is displayed: full URL, QR code, short code (e.g., `PX-A3B4C5`), copy buttons.

7.3 Broadcast to Social Channels

8. Below the link card, a "Broadcast to Channels" panel lists all of the promoter's configured social channels with platform icons and channel names.
9. Promoter selects one or more channels (e.g., "Friday Night WhatsApp Group", "Telegram VIP List", "Instagram DM List").
10. Optionally edits the default broadcast message (pre-populated with: event name, venue, date, link, and a call-to-action).
11. Clicks "Send to Selected Channels".
12. For WhatsApp and Telegram: the system opens a platform share URL in a new tab pre-composed with the message, so the promoter completes the send within the native app. For SMS: opens the `sms: URI`. For Instagram: copies the link and message to clipboard with a note to paste into the DM.
13. The system records a `PromoterLinkBroadcast` row for each channel with `broadcastAt` timestamp and platform.
14. The promoter can return to their link's detail view at any time to see click analytics segmented by channel (using the `channel` field on `PromoterMagicLink`).

7.4 Member Attribution on Click

15. A member clicks the link. The `/join/[shortCode]` middleware records a `MagicLinkClickEvent` and reads the `presetVenueld` and `presetEventDate` from the link.
16. If the member already has a different attribution for the same venue+event scope, the middleware creates a `PENDING_CHOICE` `MemberPromoterPreference` record (upsert on unique constraint) and sets `resolvedDeadlineAt = now() + 24h`.
17. The member is presented with the promoter preference prompt before entering the booking flow.
18. On choice: `BOUND` promoter is set, `boundDecayCounter = 0`. Scope-local decay counter governs commissions for all transactions in this scope. The non-selected promoter's record is set to `DECLINED`.
19. If the member proceeds without choosing (e.g., returns directly to checkout after 24h), the auto-resolve cron sets the first attributed promoter as `BOUND` via `FIRST_ATTRIBUTION`.

8. Conclusion & Next Steps

The Addendum B specification is well-structured and architecturally appropriate for the PRESTIX platform's multi-promoter requirements. The core attribution model, scope hierarchy, binding lock mechanism, and performance dashboard design are all sound and do not require structural revision.

The seven flags identified are all implementation-layer gaps — primarily around database constraints, access policy completeness, missing model fields, and an absent authorisation junction model. None represent a flaw in the overall design direction.

Immediate actions before Phase 1 begins:

- Resolve all seven flags (see Section 2). Estimated effort: 1–2 days.
- Confirm open design question resolutions (Section 6) with product stakeholders.
- Run the ZenStack schema migration and validate access policies with test suite.

Phase sequence:

- Phase 1 (Schema) → Phase 2 (Link API/UX) + Phase 4 (Nomination) in parallel → Phase 3 (Broadcast) → Phase 5 (Dashboard).

Implementation Confidence

Subject to flag resolutions, this implementation plan is assessed as the most appropriate and least complex path to delivering the expected promoter magic-link generation, venue nomination, social broadcasting, and performance dashboard outcomes within the existing ZenStack + NextJS + PostgreSQL architecture. No third-party services are required for Phases 1–4. Phase 5 adds a scheduled batch job (can run as a Vercel cron or a standalone Node process).